



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Experiment No. : 4

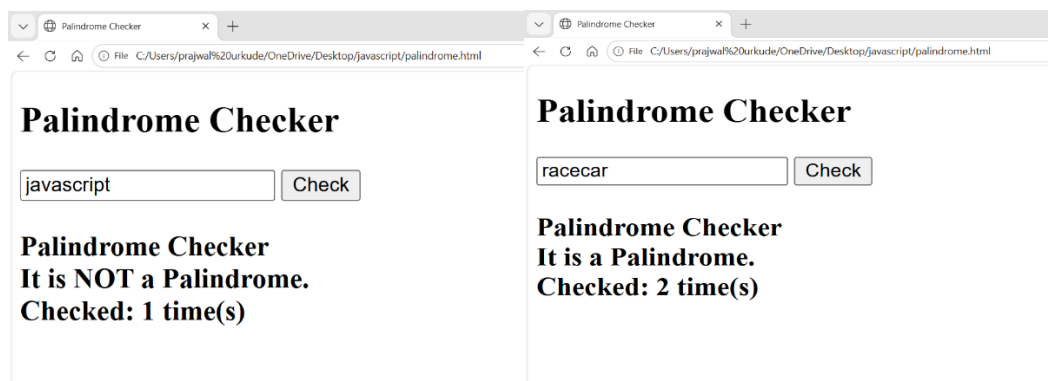
1. Title: Use function types, scope, and closures; apply try-catch; build a palindrome checker.

2. Software/Tools Required: VS code, Web browser

3. Theory:

This experiment demonstrates the use of **JavaScript functions, variable scope, closures, and exception handling** by developing a **Palindrome Checker**. Different types of functions, such as regular functions, arrow functions, and function expressions, are used to perform palindrome checking. Variable scope (var, let, and const) ensures proper accessibility of variables, while **closures** allow inner functions to access variables from their outer function even after execution. **Try-catch** blocks are used to handle invalid or unexpected inputs gracefully, preventing the program from crashing. The application checks whether the entered string is the same when read forwards and backwards and displays the result dynamically.

4. Output:



Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Palindrome Checker</title>
</head>
<body>

<h2>Palindrome Checker</h2>

<input type="text" id="text" placeholder="Enter a word">
<button onclick="checkPalindrome()">Check</button>

<h3 id="result"></h3>

<script>

  // Global Scope
  let appName = "Palindrome Checker";

  // Closure Function
  function counter() {
    let count = 0; // Local Scope

    return function () {
      count++;
      return count;
    };
  }

  const checkCount = counter();

  // Anonymous Function
  const cleanText = function (str) {
    return str.toLowerCase().replace(/[^a-z0-9]/g, "");
  };

  // Arrow Function
  const isPalindrome = (str) => {
    return str === str.split("").reverse().join("");
  };

  // Normal Function
  function checkPalindrome() {
```

```

try {

    let input = document.getElementById("text").value;

    if (input.trim() === "") {
        throw "Please enter some text!";
    }

    let text = cleanText(input);

    let result = isPalindrome(text)
        ? "It is a Palindrome."
        : "It is NOT a Palindrome.";

    document.getElementById("result").innerHTML =
        appName + "<br>" +
        result + "<br>" +
        "Checked: " + checkCount() + " time(s)";

}
catch (error) {
    document.getElementById("result").innerHTML =
        "Error: " + error;
}

}

</script>

</body>
</html>

```

5.Result/Conclusion

The palindrome checker was successfully developed using JavaScript. The experiment demonstrated the practical use of **function types, variable scope, closures, and try-catch** for exception handling. It improved understanding of reusable functions, lexical scoping, error handling, and string manipulation while creating an interactive and reliable web application.